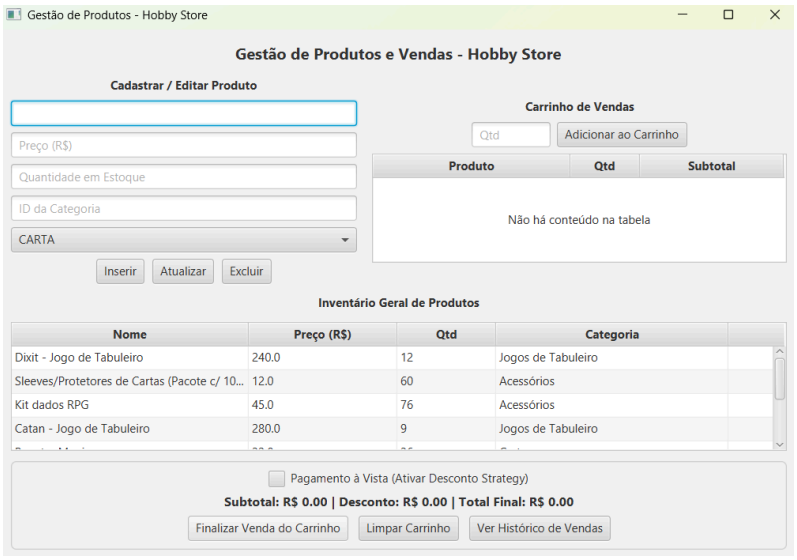
Relatório: Evolução e Arquitetura do Sistema Hobby Store

Nome: Maria E. Menon
Turma: TB

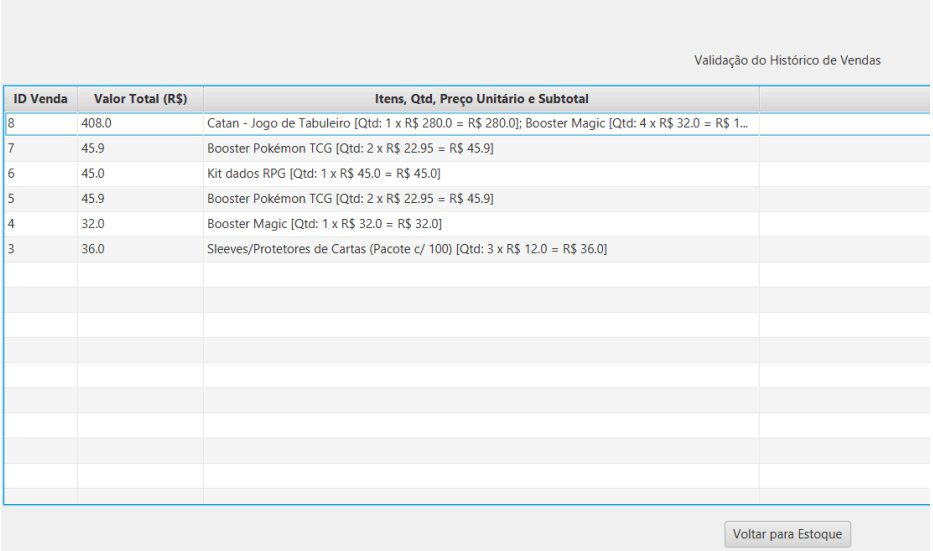




Este documento apresenta a segunda versão do sistema Hobby Store. A primeira etapa do projeto estabeleceu as fundações de armazenamento e gestão do inventário, integrando o banco de dados PostgreSQL com a aplicação Java via padrão DAO. No entanto, no antigo projeto não foi possível finalizar a interface gráfica, deixando o histórico de transações, restritas apenas ao backend e à base de dados, sem uma visualização completa.

O principal objetivo desta 2 etapa foi melhorar o sistema de uma simples aplicação CRUD para uma arquitetura orientada a objetos mais robusta, aplicando a Arquitetura em Camadas e Padrões de Projeto GoF.

A primeira grande mudança foi a finalização da funcionalidade de Histórico de Vendas. Anteriormente, o registo de transacções era limitado e não possuía representação visual completa. Nesta versão, a interface gráfica foi integrada, permitindo a consulta direta do histórico das transações:



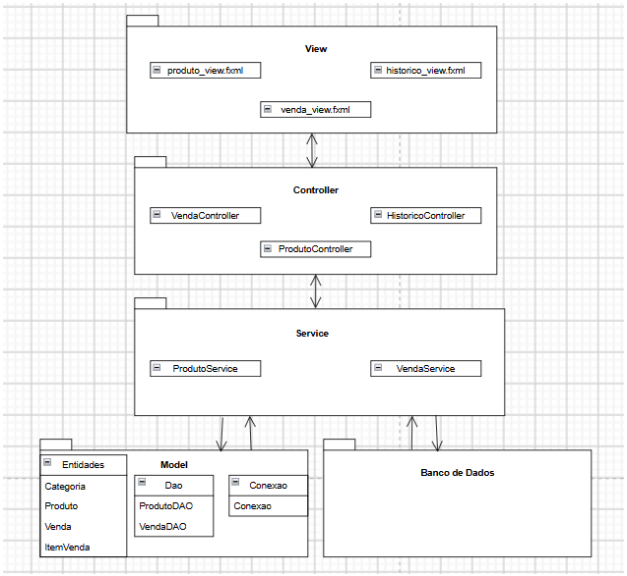
ID Venda: O identificador único de cada transação.

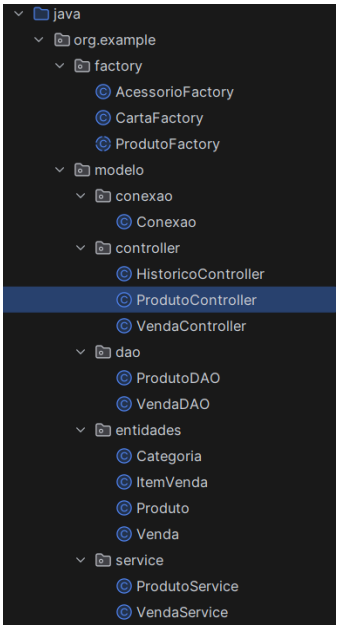
Valor Total (R$): O valor final da venda

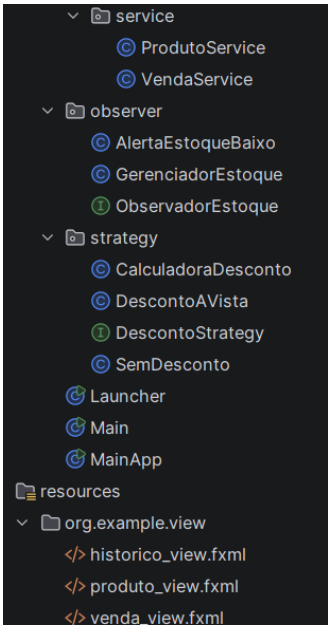
Itens, Qtd, Preço Unitário e Subtotal: O detalhamento de cada item da compra, como [Nome do Produto] [Qtd: ; R$ Preço = ; R$ Subtotal]

- **Arquitetura em Camadas**

Foi implementado a Service Layer (pelas classes ProdutoService e VendaService) como o componente intermediário na comunicação entre a interface e a base de dados. Com esta refatoração, as responsabilidades do sistema passaram a ser divididas:

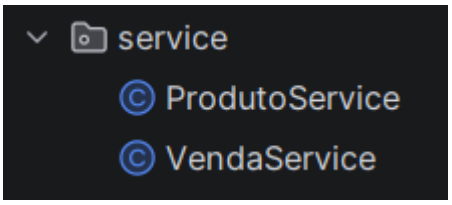






- **Padrões de Projeto GoF**

1. **Service Layer** = Como requisito obrigatório da arquitetura, a **Service Layer** foi colocada para atuar como o “núcleo de processamento”. Sem esta camada, o ProdutoController ficaria “sobrecarregado” acumulando validações e regras de



cálculo.

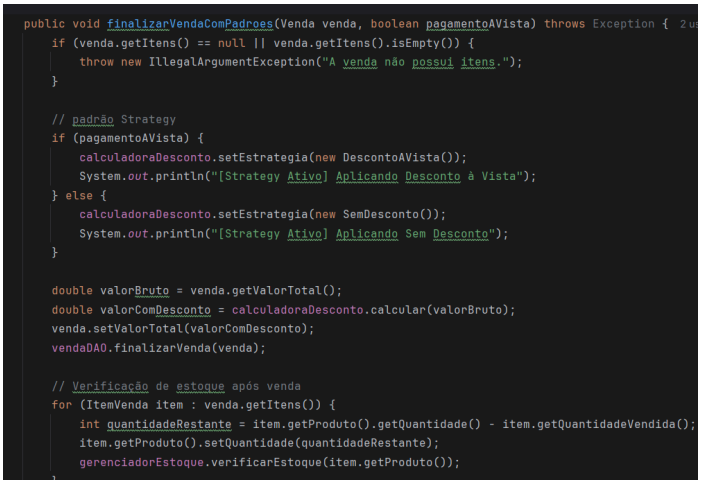
O **ProdutoService** atua como responsável por centralizar todas as regras de negócio referentes aos produtos da loja. Ela atua como intermediária entre o controlador da tela e a persistência: Recebe os dados enviados pela interface (nome, preço, quantidade, categoria); Aciona o padrão Factory Method para instanciar o tipo correto de produto (Carta, Acessorio ou Tabuleiro) ; Executa as operações de cadastro, listagem, atualização e exclusão, comunicando-se diretamente com o ProdutoDAO para gravar as alterações na base de dados.

```
public void cadastrarProduto(String tipo, String nome, double preco, int quantidade)
    ProdutoFactory factory;

    if (tipo != null && tipo.equalsIgnoreCase( anotherString: "ACESSORIO")) {
        factory = new AcessorioFactory();
        System.out.println("[Factory Method] A usar AcessorioFactory");
    } else {
        factory = new CartaFactory();
        System.out.println("[Factory Method] A usar CartaFactory");
    }

    Produto produto = factory.criarProduto(nome, preco, quantidade);
    produtoDAO.inserir(produto);
}
```

O **VendaService** é a classe responsável por concentrar toda a lógica complexa de transações e finalização de compras. As suas principais funções são: Processar os itens adicionados ao carrinho de vendas; Integrar o cálculo de preços aplicando as regras do padrão Strategy (como o desconto de pagamento à vista); Registar a venda na base de dados por meio do VendaDAO, atualizando e baixando as quantidades do stock; Disparar o padrão Observer através do GerenciadorEstoque para verificar se algum produto atingiu o limite crítico após a baixa no inventário.



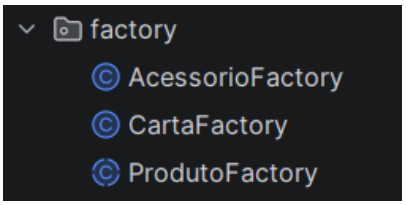
```
public void finalizarVendaComPadroes(Venda venda, boolean pagamentoAVista) throws Exception {
    if (venda.getItens() == null || venda.getItens().isEmpty()) {
        throw new IllegalArgumentException("A venda não possui itens.");
    }

    // padrão Strategy
    if (pagamentoAVista) {
        calculadoraDesconto.setEstrategia(new DescontoAVista());
        System.out.println("[Strategy Ativo] Aplicando Desconto à Vista");
    } else {
        calculadoraDesconto.setEstrategia(new SemDesconto());
        System.out.println("[Strategy Ativo] Aplicando Sem Desconto");
    }

    double valorBruto = venda.getValorTotal();
    double valorComDesconto = calculadoraDesconto.calcular(valorBruto);
    venda.setValorTotal(valorComDesconto);
    vendaDAO.finalizarVenda(venda);

    // Verificação de estoque após venda
    for (ItemVenda item : venda.getItens()) {
        int quantidadeRestante = item.getProduto().getQuantidade() - item.getQuantidadeVendida();
        item.getProduto().setQuantidade(quantidadeRestante);
        gerenciadorEstoque.verificarEstoque(item.getProduto());
```

2. **Factory Method** = O **Factory Method** foi usado para criar os produtos da loja (como Cartas, Acessórios e Jogos de Tabuleiro). Em vez de criarmos os objetos diretamente no código com new, o sistema utiliza uma classe fábrica (ProdutoFactory) que recebe a categoria escolhida e decide qual o objeto exato que deve ser instanciado de forma automática e organizada.



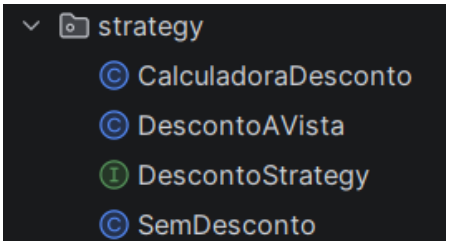
O **AcessorioFactory** é a classe central que o ProdutoService chama quando a interface pede para cadastrar um produto. Ela analisa a opção selecionada no menu (CARTA, ACESSORIO ou TABULEIRO) e decide como o objeto será criado. Se for uma carta, ela delega para a CartaFactory; se for um acessório, para a AcessorioFactory; e se for um tabuleiro, ela própria faz a criação direta

O **CartaFactory** é a classe responsável exclusivamente por instanciar produtos da categoria de cartas. Ela garante que todos os dados específicos de cartas sejam formatados e atribuídos corretamente antes de entregar o objeto pronto para o sistema.

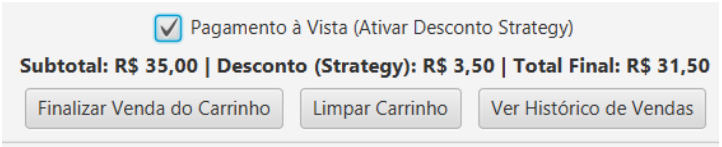
O **ProdutoFactory** é a classe responsável por instanciar os itens da categoria de acessórios (como protetores de cartas/sleeves, caixas de deck, dados e escudos). Ela isola qualquer regra específica de montagem dessa categoria.

Não foi implementada uma classe TabuleiroFactory porque a sua criação não justificava a existência de uma estrutura própria. Como a CartaFactory e a AcessorioFactory servem para instanciar os tipos específicos (id1 e Id2), qualquer produto que não se enquadre nesses dois é um tabuleiro

3. **Strategy** = O **Strategy** foi usado para calcular o desconto no momento da venda. Coloquei que, se o pagamento for à vista, o sistema aplica um desconto de 10%, e se não for, aplica a regra de sem desconto. O padrão serve para separar essa lógica de cálculo em classes diferentes, o que permite alternar a forma como o preço é calculado.







O **CalculadraDesconto** é a interface que define o contrato base para o cálculo dos descontos, estabelecendo o método comum que todas as regras de pagamento devem implementar.

```java
package org.example.strategy;

public class CalculadoraDesconto {  6 usages
    private DescontoStrategy estrategia;  3 usages

    // qual estratégia será usada no  cálculo
    public void setEstrategia(DescontoStrategy estrategia) {  4 usages
        this.estrategia = estrategia;
    }

    public double calcular(double valorOriginal) {  2 usages
        if (estrategia == null) {
            return valorOriginal; // Se nenhuma estratégia foi definida, retorna o valor normal
        }
        return estrategia.calcularPrecoFinal(valorOriginal);
    }
}
```

O **DescontoAVista** é a classe concreta da estratégia que aplica o abatimento percentual sobre o valor total do pedido quando a opção de pagamento à vista é selecionada.

```java
package org.example.strategy;

public class DescontoAVista implements DescontoStrategy {  4 usages
    @Override  1 usage
    public double calcularPrecoFinal(double valorOriginal) {
        // Aplica 10% de desconto para pagamentos à vista
```

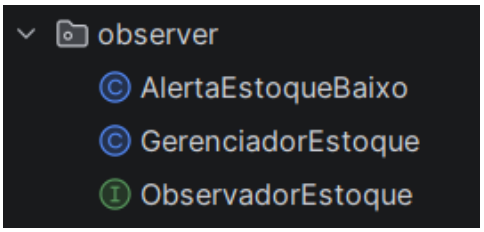
O **SemDesconto** é a classe concreta da estratégia que mantém o valor integral da compra, sendo utilizada quando a condição de pagamento escolhida não possui benefício de desconto.

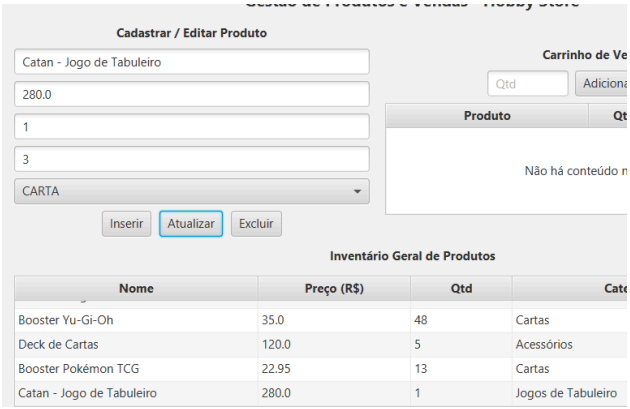
```java
package org.example.strategy;

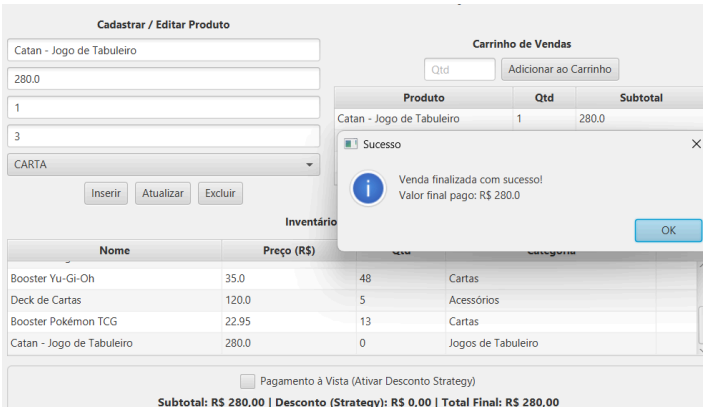
public class SemDesconto implements DescontoStrategy {  4 usages
    @Override  1 usage
    public double calcularPrecoFinal(double valorOriginal) {
        return valorOriginal; // Sem alteração no valor
    }
}
```

O **DescontoStrategy** é a classe de contexto que recebe a estratégia de desconto a ser utilizada e executa o cálculo final

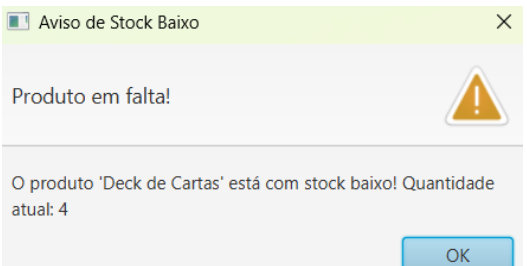
4. **Observer** = O **Observer** foi usado para monitorar o stock do inventário automaticamente. Sempre que uma venda é finalizada e o produto sofre baixa, o sistema avisa os observadores para verificar se a quantidade chegou a níveis críticos e disparar o alerta de stock baixo.







O **AlertarEstoqueBaixo** *é a classe concreta que implementa a interface ObservadorEstoque, sendo responsável por receber o aviso e disparar a notificação visual ou o alerta quando a quantidade de um produto atinge o limite mínimo.*



O **GerenciadorEstoque** é a classe subject que mantém a lista de observadores registados, controlando o estado do stock

O **ObservadorEstoque** é a interface que define o contrato base para os observadores, estabelecendo o método de notificação que deve ser implementado por qualquer classe interessada em monitorar as alterações de stock.

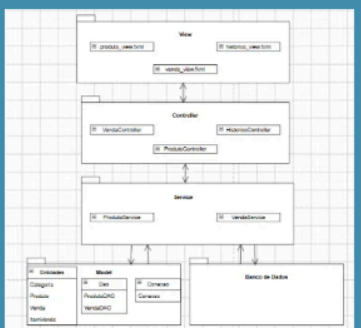
Slides da Apresentação:

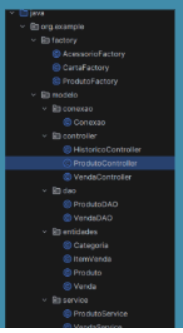


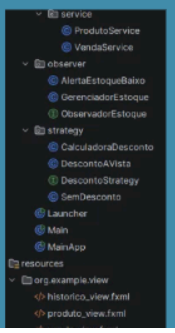


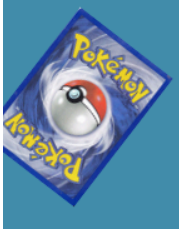
Organização do Sistema

A organização do código da Hobby Store foi estruturada com a implementação do Service Layer, estabelecendo uma divisão de responsabilidades na arquitetura do sistema. Com isso, o código evita misturar a interface com as regras operacionais e o acesso a dados, organizando-se em frentes distintas onde a camada de apresentação foca na interação com o utilizador, o Service Layer centraliza toda a lógica de negócio, regras de validação e processamentos das operações da loja, e a persistência gere o armazenamento de forma isolada.











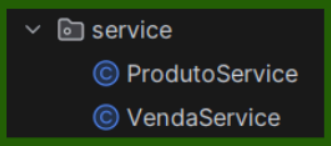




Service Layer



No contexto arquitetural do sistema, o Service Layer tem como principal objetivo atuar como uma camada intermediária de isolamento, garantindo que as regras de negócio e os processos operacionais fiquem completamente separados da interface gráfica e do acesso direto a dados. Dentro dessa estratégia de organização, destacam-se as classes ProdutoService e VendasService



A classe ProdutoService tem como objetivo central gerir toda a lógica de negócio e as operações associadas aos produtos do sistema, funcionando como a ponte entre os controladores da interface e a persistência de dados.



ProdutoService e VendasService

No contexto arquitetural do sistema, o Service Layer tem como principal objetivo atuar como uma camada intermediária de isolamento, garantindo que as regras de negócio e os processos operacionais fiquem completamente separados da interface gráfica e do acesso direto a dados. Dentro dessa estratégia de organização, destacam-se as classes ProdutoService e VendasService

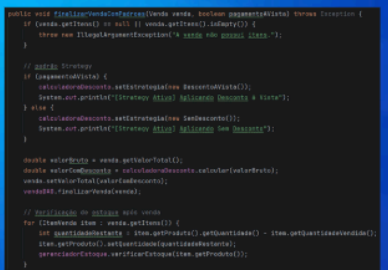
- **ProdutoService**: Gere o ciclo de vida completo dos produtos, lidando com a listagem, validação de IDs em atualizações ou exclusões, e coordenando o processo de cadastro dinâmico por meio de fábricas especializadas.

```
public void cadastrarProduto(String tipo, String nome, double preco, int quantidade) {
    ProdutoFactory factory;

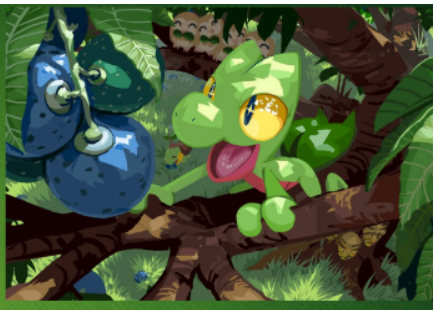
    if (tipo != null && tipo.equalsIgnoreCase( anotherString: "ACESSORIO")) {
        factory = new AcessorioFactory();
        System.out.println("[Factory Method] A usar AcessorioFactory");
    } else {
        factory = new CartaFactory();
        System.out.println("[Factory Method] A usar CartaFactory");
    }

    Produto produto = factory.criarProduto(nome, preco, quantidade);
    produtoDAO.inserir(produto);
}
```

- **VendasService**: Valida a integridade do carrinho de compras, aplica as estratégias de cálculo de desconto conforme a tipo de pagamento, e interage com o gestor de stock para acionar os observadores de monitorização.



Padrões de Projeto GoF



Para atender aos requisitos do padrões de projeto fundamentais do GoF., optou-se pela aplicação do: Factory Method que foi escolhido para padronizar e encapsular a criação de novos produtos no sistema, o Strategy para gerir o cálculo de descontos, e o Observer foi implementado para monitorar o stock em tempo real e disparar alertas automáticos sempre que necessário.



- **Factory Method:** Tem como classes ProdutoFactory: A interface que define o contrato de criação, AcessorioFactory e CartaFactory: As fábricas concretas responsáveis por instanciar respetivamente os acessórios e as cartas de forma isolada.
- **Strategy:** Tem como classes a CalculadoraDesconto (o contexto que aplica a estratégia), a interface/classe base DescontoStrategy, e as implementações concretas como DescontoAVista e SemDesconto
- **Observer:** Tem como classes GerenciadorEstoque (o sujeito observado), a interface/classe ObservadorEstoque, e a classe concreta AlertaEstoqueBaixo (o observador que dispara os avisos)

Factory Method

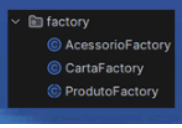
Antes da refatoração, a criação de objetos dependia de blocos condicionais (if/else ou switch) com instanciações feitas com o new espalhado pelas classes de serviço. Sempre que um novo tipo de item entrava no sistema, o código precisava de modificações, o que aumentava o acoplamento e tornava a manutenção mais complexa. Após o Factory Method a lógica de instanciação passa a ser tratada pelas fábricas centralizadas (CartaFactory e AcessorioFactory), enquanto produtos do tipo Tabuleiro são instanciados diretamente no ProdutoService, por não exigirem a complexidade de uma fábrica dedicada.

- ProdutoFactory: É a interface que define o contrato de criação para as fábricas concretas. No código, ela garante que qualquer fábrica que a implemente siga o mesmo padrão obrigatório.

```java
package org.example.factory;

import org.example.modelo.entidades.Produto;

public abstract class ProdutoFactory {
    // que as subclasses vão implementar
    public abstract Produto criarProduto(String nome, double preco, int quantidade);
}
```



- CartaFactory e AcessorioFactory: Fábricas concretas que implementam o contrato e isolam a lógica de construção específica de cartas e acessórios.

```java
public class CartaFactory extends ProdutoFactory {
    @Override
    public Produto criarProduto(String nome, double preco, int quantidade) {
        Produto produto = new Produto();
        produto.setNome(nome);
        produto.setPreco(preco);
        produto.setQuantidade(quantidade);

        // Define automaticamente a categoria de cartas (ex: ID 1 para TCG)
        Categoria categoria = new Categoria();
        categoria.setId(1);
        categoria.setNome("Cartas TCG");
        produto.setCategoria(categoria);

        return produto;
    }
}
```

```java
public class AcessorioFactory extends ProdutoFactory {
    @Override
    public Produto criarProduto(String nome, double preco, int quantidade) {
        Produto produto = new Produto();
        produto.setNome(nome);
        produto.setPreco(preco);
        produto.setQuantidade(quantidade);

        // Define automaticamente a categoria de acessórios (ex: ID 2 para Acess
        Categoria categoria = new Categoria();
        categoria.setId(2);
        categoria.setNome("Acessórios");
        produto.setCategoria(categoria);

        return produto;
    }
}
```



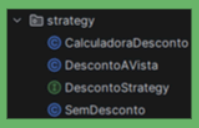
Strategy

O padrão Strategy foi implementado para gerir as regras de desconto no sistema de vendas. Quando o cliente opta por realizar um pagamento à vista, o sistema aplica automaticamente um desconto de 10%. Além disso, essa modalidade fica marcada e registada diretamente no histórico de vendas da compra.

- DescontoStrategy: Interface que define o contrato padrão para os algoritmos de desconto, garantindo que todas as regras sigam a mesma "assinatura"

```java
package org.example.strategy;

public interface DescontoStrategy {
    double calcularPrecoFinal(double valorOriginal);
}
```

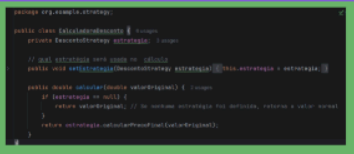


- DescontoAVista: Estratégia que aplica o cálculo específico dos 10% de desconto quando o pagamento é feito à vista.

```java
package org.example.strategy;

public class DescontoAVista implements DescontoStrategy {
    @Override
    public double calcularPrecoFinal(double valorOriginal) {
        // 10%
        return valorOriginal * 0.90;
    }
}
```

- CalculadoraDesconto: Classe responsável por receber a forma de pagamento escolhida e delegar a execução do cálculo para a estratégia correspondente.



- SemDesconto: Estratégia concreta aplicada quando não há desconto elegível para a transação.

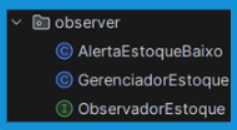
```java
package org.example.strategy;

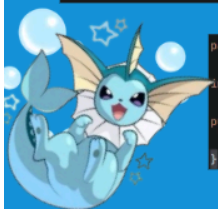
public class SemDesconto implements DescontoStrategy {
    @Override
    public double calcularPrecoFinal(double valorOriginal) {
        return valorOriginal; // Sem alteração no valor
    }
}
```

Observer

O padrão Observer foi implementado para automatizar o monitoramento do stock de produtos. Sempre que ocorre uma alteração relevante no inventário (algum item com 5 ou menos), o sistema notifica

- ObservadorEstoque: A interface que define o contrato padrão para os elementos que desejam ser notificados sobre as alterações no stock.





```java
package org.example.observer;

import org.example.modelo.entidades.Produto;

public interface ObservadorEstoque {  5 usages
    void atualizar(Produto produto);  1 usage
}
```

- GerenciadorEstoque: A classe principal (sujeito) que controla o inventário, gere a lista de observadores registados e dispara os avisos sempre que o estado do stock muda.

```java
public class GerenciadorEstoque {  3 usages
    private List<ObservadorEstoque> observadores = new ArrayList<>();

    public void adicionarObservador(ObservadorEstoque observador) {
        observadores.add(observador);
    }

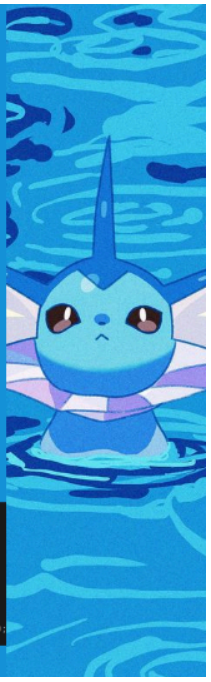
    public void removerObservador(ObservadorEstoque observador) {
        observadores.remove(observador);
    }

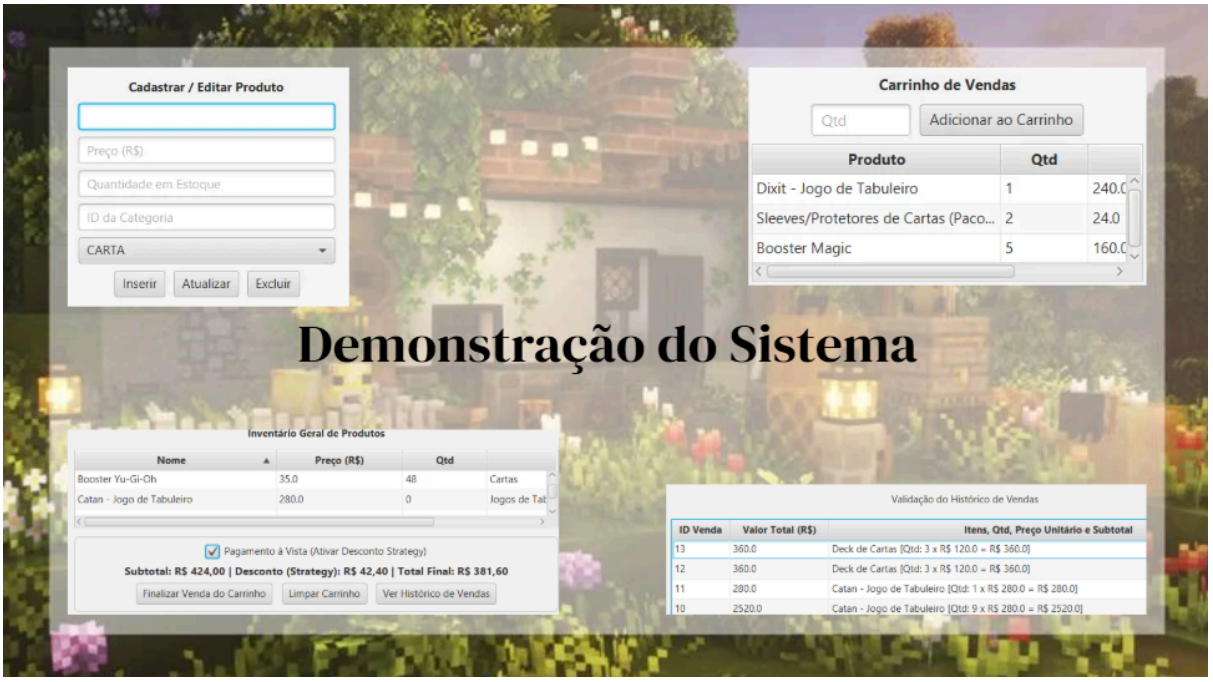
    public void verificarEstoque(Produto produto) {  1 usage
        // Se o stock estiver menor ou igual a 5 unidades, dispara o
        if (produto.getQuantidade() <= 5) {
            for (ObservadorEstoque obs : observadores) {
                obs.atualizar(produto);
            }
```

- AlertaEstoqueBaixo: A classe concreta que implementa o observador para detetar quando um item atinge níveis mínimos, despoletando o aviso de stock baixo.

```java
public class AlertaEstoqueBaixo implements ObservadorEstoque {  2 usages
    @Override  1 usage
    public void atualizar(Produto produto) {

        Platform.runLater(() -> {
            Alert alert = new Alert(Alert.AlertType.WARNING);
            alert.setTitle("Aviso de Stock Baixo");
            alert.setHeaderText("Produto em falta!");
            alert.setContentText("O produto '" + produto.getNome() +
                    "' está com stock baixo! Quantidade atual: " + produto.getQuantidade());
            alert.showAndWait();
```




Cadastrar / Editar Produto
Preço (R$)
Quantidade em Estoque
ID da Categoria
CARTA
Inserir
Atualizar
Excluir
Carrinho de Vendas
Qtd
Adicionar ao Carrinho
Produto
Qtd
Dixit - Jogo de Tabuleiro
1
Sleeves/Protetores de Cartas (Paco...
2
24.0
Booster Magic
5
Demonstração do Sistema
Inventário Geral de Produtos
Nome
Preço (R$)
Qtd
Booster Yu-Gi-Oh
35.0
48
Cartas
Catan - Jogo de Tabuleiro
280.0
0
Pagamento à Vista (Ativar Desconto Strategy)
Subtotal: R$ 424,00 | Desconto (Strategy): R$ 42,40 | Total Final: R$ 381,60
Finalizar Venda do Carrinho
Limpar Carrinho
Ver Histórico de Vendas
Validação do Histórico de Vendas
ID Venda
Valor Total (R$)
Itens, Qtd, Preço Unitário e Subtotal
13
360.0
Deck de Cartas [Qtd: 3 x R$ 120.0 = R$ 360.0]
12
360.0
Deck de Cartas [Qtd: 3 x R$ 120.0 = R$ 360.0]
11
280.0
Catan - Jogo de Tabuleiro [Qtd: 1 x R$ 280.0 = R$ 280.0]
10
2520.0
Catan - Jogo de Tabuleiro [Qtd: 9 x R$ 280.0 = R$ 2520.0]


fIM
OBRIGADA!
MARIA EDUARDA MENON
PROGRAMAÇÃO ORIENTADA A OBJETOS II